

PART II — BUILDING THE INGESTION PIPELINE

Chapter 6

When RAG Is the Wrong Tool: The Two-Track Problem

"The right tool for the wrong job is still the wrong tool."

Chapter 6 — When RAG Is the Wrong Tool: The Two-Track Problem

Chapter 5 left you with a pipeline capable of loading documents from any source. You can feed it PDFs, text files, Markdown pages, and — as you saw in the final section — CSV and Excel files. The CSV loader works. The rows ingest. The embeddings are generated. So when your user types *"What is the average age of patients in the dataset?"* the system should answer correctly, right?

It will not. It will return a paragraph of text that mentions age — and it will do so confidently, because the retriever found chunks that are semantically close to the word *"average age"*. But it will not return 34.7. It will not return any number at all. This is not a bug in your implementation; it is a fundamental mismatch between what similarity search does and what aggregation requires. This chapter explains that mismatch, names it, and gives you the architecture to fix it.

By the time you finish Section 6.6, you will have a clear mental model of when CSV data belongs in a vector store and when it belongs in a pandas DataFrame — and a working skeleton for each path. The two-track architecture you build here is the foundation for every structured-data capability in the book that follows.

6.1 The CSV-Stats Trap — Why “Give Me the Average Age” Can’t Be Answered by Similarity Search

Imagine a dataset of two thousand patient records, each row containing fields like *patient_id*, *age*, *ethnicity*, and *diagnosis*. You ingest this CSV with Chapter 5's loader. Each row becomes a LangChain Document whose *page_content* looks something like: *"age: 34, ethnicity: Caucasian, diagnosis: ASD"*. The embedding model encodes that text and stores it in ChromaDB. Everything looks fine.

Now the user asks: *"What is the average age of Caucasian patients?"* The retriever embeds this query, searches ChromaDB, and finds the *k* chunks that are most semantically similar to the phrase *"average age Caucasian"*. It returns five or ten rows that happen to mention Caucasian patients. The LLM reads those rows and either hallucinates a number, says it cannot compute an average from the context, or guesses wildly. In no case does it return the correct mean across all two thousand rows — because the retriever never saw all two thousand rows. It only saw the ten most similar ones.

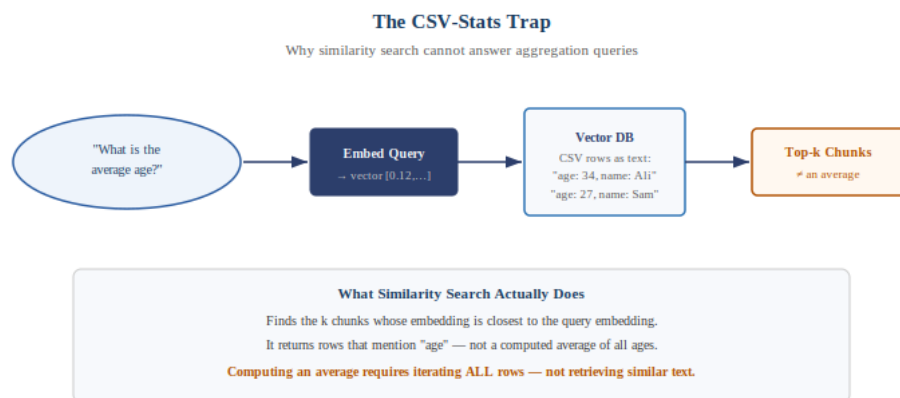


Figure 6.1 — The CSV-stats trap. The retriever returns the most similar rows, not a computed aggregate. No matter how many chunks are retrieved, similarity search cannot replace a full-table scan.

The root of the problem is that similarity search and aggregation are categorically different operations. Similarity search says: *"find me the k things most similar to this."* Aggregation says: *"scan everything, apply a function, return one number."* These two operations cannot substitute for each other, any more than a library index card can add up the page counts in a collection.

Definition — The CSV-stats trap

The CSV-stats trap occurs when a RAG system is asked an aggregation query (count, sum, average, maximum, minimum, group-by) over structured tabular data. Because the retriever returns only the k most similar chunks, it never accesses all rows — making exact computation impossible. The result is either a hallucinated number or a retrieval answer that describes individual records rather than computes across them.

The failure is invisible unless you test for it. The system does not throw an exception. It does not say *"I cannot compute averages"*. It returns a fluent answer backed by real retrieved text, which is exactly what makes the trap dangerous. The remedy is not to improve the retriever; it is to route the query to a different tool entirely.

6.2 Semantic Retrieval vs. Aggregation — A Fundamental Mismatch

To understand why the fix requires an architectural change rather than a prompt tweak, you need to see the two operations side by side. Semantic retrieval and aggregation are not two points on a spectrum of the same technique; they are entirely different computational processes that happen to look superficially similar because both accept a natural-language query and return an answer.

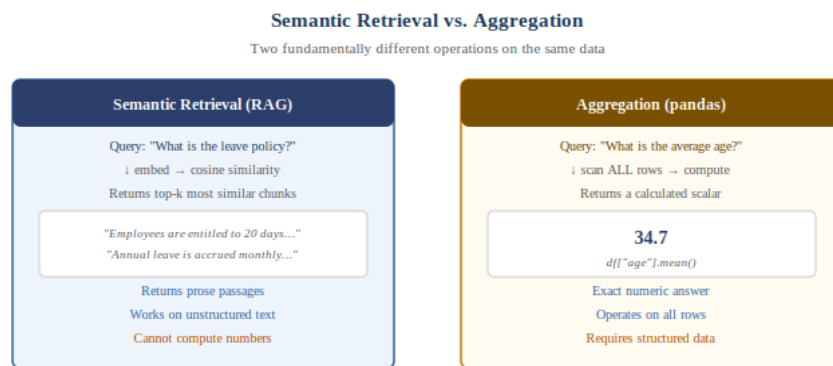


Figure 6.2 — Semantic retrieval versus aggregation. The left panel shows what a RAG retriever actually does; the right panel shows what a pandas call does. The inputs look similar; the mechanisms are entirely different.

The table below makes the distinction explicit across the dimensions that matter most for system design:

Dimension	Semantic Retrieval	Aggregation
Operation	Find k-nearest vectors	Scan all rows, apply function
Data requirement	Text corpus, embedded	Structured table, typed columns
Output	Relevant text passages	Scalar, series, or small table
Scalability	Sub-linear (ANN index)	Linear (full-table scan)
Handles missing rows	Yes (approximate by design)	No (incomplete scan = wrong answer)
Suitable for	"Explain X", "Find docs about Y"	"Average of", "Count where", "Sum of"
Failure mode	Irrelevant passage returned	Hallucinated or incomplete number

The critical row is the last one. When semantic retrieval fails, the LLM usually signals uncertainty or returns a passage that misses the question. When an aggregation query is routed through RAG, the LLM fabricates a number that is plausible but wrong — and does so confidently. That asymmetry of failure modes is why architectural separation is not optional.

Analogy — The card catalogue vs. the accountant

A library card catalogue is perfect for finding books about a topic. Ask it *"books about penicillin"* and it finds them. Ask it *"how many books do we own?"* and it is useless — not because the catalogue is broken, but because counting requires walking every shelf, not searching an index. Semantic retrieval is the card catalogue. Aggregation is the accountant who physically counts every item on every shelf. The right question for the catalogue and the right question for the accountant are categorically different. Giving the aggregation question to the catalogue does not produce a wrong answer from the catalogue — it produces the right catalogue answer to the wrong question.

This is also why larger context windows or better retrievers do not solve the problem. Even if you retrieved all two thousand rows into the context window, the LLM would still need to compute a mean over those rows — an arithmetic operation that language models perform unreliably when the number of rows is large. The correct solution does not push the computation onto the LLM; it routes the computation to a tool that was designed for it: pandas.

6.3 The Two-Track Architecture — RAG for Prose, pandas for Structured Data

The two-track architecture is simple to state: prose queries go to RAG; aggregation queries go to pandas. The implementation requires three additions to the pipeline you already have: a router that classifies each incoming query, a pandas track that loads the CSV into a DataFrame and executes queries against it, and a result formatter that wraps the pandas output into a natural-language answer. The RAG track remains unchanged.

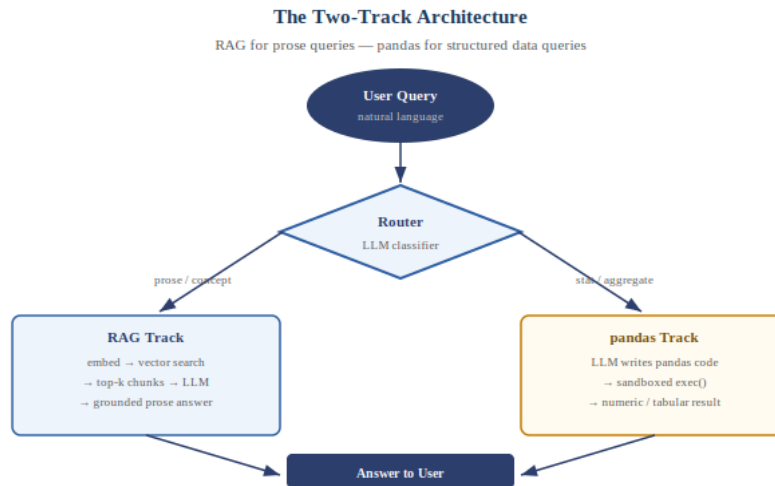


Figure 6.3 — The two-track architecture. Every query is routed at the entry point. RAG handles prose and concept questions; the pandas track handles aggregation and structured-data queries. The two tracks share no components — they are independent execution paths.

The key design principle is that the two tracks are independent. The vector store and the DataFrame are separate data structures loaded from the same source files but optimised for different query types. A CSV file is loaded twice: once by the ingestion pipeline (to generate text embeddings for row-level lookup), and once into a pandas DataFrame (to support aggregation queries). This duplication is intentional and cheap — a five-megabyte CSV produces perhaps fifty megabytes of embeddings but loads into pandas in under a second.

The ingestion side has already been handled by Chapter 5. On the pandas side, loading is straightforward:

```
# query.py — load CSV data for the pandas track
import pandas as pd
import glob

def load_csv_dataframes(csv_dir: str) -> pd.DataFrame | None:
    csv_files = glob.glob(f"{csv_dir}/*.csv")
    if not csv_files:
        return None
    frames = [pd.read_csv(f) for f in csv_files]
    df = pd.concat(frames, ignore_index=True)
    print(f"CSV track loaded — {len(df)} rows, columns: {list(df.columns)}")
    return df
```

If your data directory contains multiple CSV files, `pd.concat` merges them into one DataFrame. This works correctly when the files share the same column schema — the common case when multiple batches of records are stored as separate files. If the CSVs have different schemas, load them into separate DataFrames and route queries to the correct one based on the columns referenced in the query.

Common pitfall — Forgetting to remove CSVs from the ingestion search roots

If your `SEARCH_ROOTS` list in `ingest.py` includes the `data/csv` directory, every CSV will also be ingested into the vector store as text chunks. The pandas track will then co-exist with a set of row-level text chunks in ChromaDB. For most aggregation queries this is harmless — the router will send them to pandas anyway — but for large CSVs it wastes significant embedding and storage resources. Unless you explicitly want row-level lookup (covered in Section 6.6), remove CSVs from the ingestion pipeline once the two-track architecture is in place.

6.4 Routing the User's Query to the Right Track

The router sits at the entry point of the query pipeline. Its job is binary: given a query string, decide whether to send it to the RAG track or the pandas track. Two routing strategies are practical: keyword-based routing and LLM-based classification. They differ in speed, accuracy, and the types of queries they handle well.

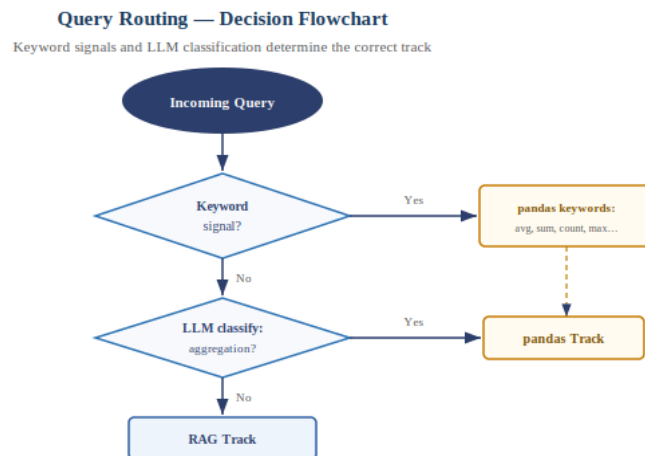


Figure 6.4 — Query routing decision flowchart. Keyword signals are checked first because they are fast and reliable. Ambiguous queries that pass the keyword filter are then classified by the LLM. The RAG track is the default when neither signal fires.

The simplest router checks the query for a vocabulary of aggregation-intent words. These words are almost never used in prose retrieval queries, so false positives are rare:

```
# query.py - keyword router
CSV_KEYWORDS = {
    "average", "mean", "median", "std", "variance",
    "sum", "total", "count", "how many", "percentage",
    "minimum", "maximum", "min", "max", "distribution",
    "correlation", "histogram", "group by", "breakdown",
}

def is_csv_query(query: str) -> bool:
    q = query.lower()
    return any(kw in q for kw in CSV_KEYWORDS)
```

Keyword routing catches the majority of numerical and statistical queries cleanly and cheaply — no LLM call required. Its weakness is queries that use natural phrasing without trigger words: *"Tell me the typical age of a patient in this dataset"* does not contain *"average"* or *"mean"*, but it is clearly an aggregation query. This is where LLM-based classification adds value.

For queries that pass the keyword filter, the LLM can classify intent with high accuracy using a short classification prompt. The classification is a separate, inexpensive LLM call — one that does not retrieve any chunks:

```
# query.py - LLM-based query classifier
CLASSIFY_PROMPT = """You are a query classifier for a data system.
Given a user question, decide if it requires computing a statistic or
aggregate from a structured table (answer: 'aggregation'), or if it
asks for information that would be found in a text document
(answer: 'retrieval').
Respond with exactly one word: 'aggregation' or 'retrieval'.

Question: {query}"""

def classify_query(query: str, llm) -> str:
    prompt = CLASSIFY_PROMPT.format(query=query)
    result = llm.invoke(prompt).content.strip().lower()
    return result # 'aggregation' or 'retrieval'

def route_query(query: str, df, llm) -> str:
    if df is None:
        return 'rag'
    if is_csv_query(query):
        return 'pandas'
    label = classify_query(query, llm)
    return 'pandas' if label == 'aggregation' else 'rag'
```

Definition — Query router

A query router is a lightweight classification component that inspects an incoming natural-language query and dispatches it to the appropriate execution track. In the two-track architecture, it sends aggregation-intent queries to the pandas track and prose-intent queries to the RAG track. The router is the single decision point that keeps the two tracks independent.

The combined approach — keyword check first, LLM fallback second — gives you the best of both: fast routing for the common cases, accurate routing for the edge cases. The LLM classification call costs approximately the same as generating twenty tokens, which is negligible relative to the full RAG pipeline.

6.5 LLM-Generated pandas Code as a Sandboxed Tool

Once a query is routed to the pandas track, the system needs to translate the natural-language query into a pandas expression and execute it. The cleanest way to do this is to ask the LLM to write the code, then run that code in a restricted execution sandbox. This approach handles the full range of pandas operations — filtering, grouping, sorting, cross-tabulating — without requiring you to hand-craft a mapping from every possible query pattern to a fixed set of operations.

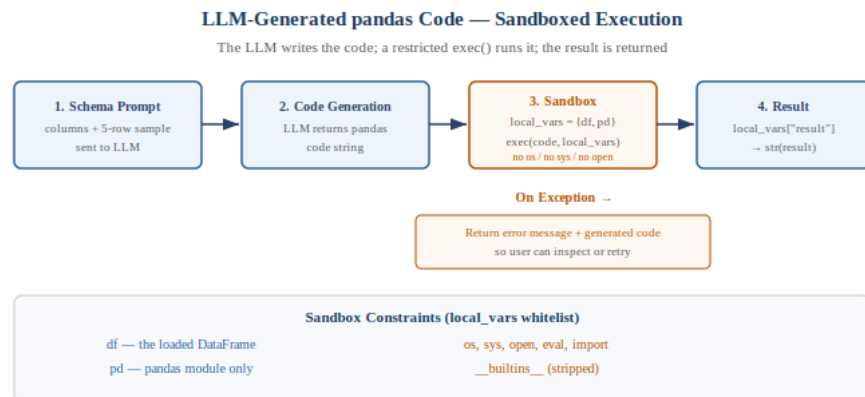


Figure 6.5 — LLM-generated pandas code execution pipeline. The schema and a small sample are sent to the LLM; the LLM returns code; the code runs in a sandbox with no access to the file system or network; the result is extracted from local_vars and returned to the user.

The code-generation prompt needs to give the LLM two things: the schema of the DataFrame (column names and types) and a small sample of real data (five rows is sufficient). Without the sample, the LLM may use column names incorrectly or assume wrong data types. With too large a sample, you waste tokens and risk hitting context limits on very wide tables.

```
# query.py - pandas code generation
PANDAS_PROMPT = """You are a data analyst. A pandas DataFrame called 'df'
has the following columns: {columns}

Sample data (first 5 rows):
{sample}

Write Python code using pandas to answer this question: {query}

Rules:
- Store the final answer in a variable called 'result'
- Use only pandas (pd) and the DataFrame (df)
- Do NOT import anything else
- Do NOT use print()
- Return ONLY the code, no explanation, no markdown fences
"""

def generate_pandas_code(query: str, df: pd.DataFrame, llm) -> str:
    prompt = PANDAS_PROMPT.format(
        columns=list(df.columns),
        sample=df.head(5).to_string(index=False),
        query=query,
    )
    raw = llm.invoke(prompt).content.strip()
    # Strip markdown fences if the model added them anyway
    return raw.replace('```python', '').replace('```', '').strip()
```

Running LLM-generated code in a production system requires a sandbox. The minimum viable sandbox for this use case is a restricted `exec()` call: you pass a `local_vars` dictionary that contains only the objects you want the code to access, and you block access to Python's built-in functions that could cause harm — file I/O, network calls, subprocess execution, and dynamic import.

```
# query.py - sandboxed executor
def execute_pandas_code(
    code: str,
    df: pd.DataFrame,
) -> str:
    local_vars = {
        'df': df,
        'pd': pd,
        '__builtins__': {}, # strip all built-ins
    }
    try:
        exec(code, {'__builtins__': {}}, local_vars)
        result = local_vars.get('result', None)
        if result is None:
            return "Code executed but 'result' was not set."
        return str(result)
    except Exception as exc:
        return f"Error: {exc}\nGenerated code was:\n{code}"

def answer_csv_query(
    query: str,
    df: pd.DataFrame,
    llm,
) -> str:
    code = generate_pandas_code(query, df, llm)
    return execute_pandas_code(code, df)
```

Common pitfall — Forgetting to strip `__builtins__` from both argument positions

Python's `exec(code, globals_dict, locals_dict)` uses the `globals_dict` for built-in lookups, not only the `locals_dict`. Passing `{'__builtins__': {}}` in `local_vars` but leaving the first argument as the real `globals()` gives the generated code full access to `open()`, `os`, and `subprocess`. Always pass `{'__builtins__': {}}` as the first argument to `exec()` as shown above.

The `execute_pandas_code` function returns the stringified result on success and a diagnostic message on failure. When a failure occurs, the full generated code is included in the response — this is intentional. It allows you to inspect what the LLM produced, correct the prompt if needed, and feed the error back in a retry loop. Error transparency beats silent failure every time.

6.6 When to Keep CSVs in RAG Anyway (Lookups, Narrow Joins, Denormalized Rows)

The two-track architecture is not a declaration that CSVs never belong in a vector store. It is a recognition that aggregation queries do not belong there. There are three patterns where embedding CSV rows directly into the vector store is not only appropriate but preferable: point lookups, narrow joins, and denormalized product or reference rows.

A point lookup is a query that asks for information about a specific, named entity: *"What is the dosage for metformin?"* or *"Show me the contract terms for supplier ID 4471."* These queries have a known answer in a specific row, and similarity search finds that row reliably. No aggregation is involved. The row contains prose-like text — drug descriptions, contract clauses, product specifications — that is semantically meaningful in vector space.

For point lookups, embedding CSV rows as Documents works well because the query is asking *"find the row about X"* rather than *"compute something across all rows"*. The vector store is exactly the right tool.

A narrow join is a query that combines information from a small number of records: *"Compare the Q1 figures for the North and South regions."* This retrieves two rows, not two thousand. When the join is narrow — when the number of rows involved is small enough that top-k retrieval can be expected to include all of them — similarity search handles it correctly.

The practical threshold is roughly $k = 5$ to $k = 20$. If your query needs data from more rows than your top-k retrieval will return, the narrow join pattern breaks down and you are back in aggregation territory.

Product catalogs, drug formularies, parts lists, and similar reference tables often have rows that are self-contained, human-readable descriptions rather than numeric measurements. A row like *"Product: noise-cancelling headphones; Features: 40-hour battery, ANC, USB-C; Price: 279"* is semantically rich enough to respond well to similarity search. A user query like *"What headphones have ANC and USB-C charging?"* will retrieve exactly the right rows via cosine similarity.

These denormalized rows behave like short prose documents. Embedding them into the vector store makes the RAG system a product search engine without any additional engineering.

The table below summarises when to use each track:

CSV Content Type	Typical Query Pattern	Recommended Track
Numeric measurements	Average, sum, count, trend	pandas track
Time-series records	Rate of change, running total	pandas track
Large survey data	Distribution, percentile	pandas track
Named-entity records	Lookup by ID or name	RAG track (embed rows)
Narrow comparative rows	Compare two or three records	RAG track (embed rows)
Denormalized reference	Search by feature or description	RAG track (embed rows)
Mixed content	Depends on query type	Router decides at runtime

Analogy — The specialist and the generalist

Think of the two tracks as two colleagues. The generalist (RAG) has read everything and can answer any conceptual question from memory. The specialist (pandas) has access to a live database and can compute any number on demand. Most questions go to one or the other naturally. A question about what the leave policy says goes to the generalist. A question about how many employees took leave in Q3 goes to the specialist. Occasionally a question could go either way — and that is exactly what the router is for: making a fast, sensible decision so neither colleague is given a task they cannot perform well.

One final note on the mixed-content case. Some CSV files contain both prose columns (description, notes, comments) and numeric columns (age, count, revenue). For these files, the router is doing the right work at query time: prose questions are sent to RAG, where the text columns are indexed and retrievable; numeric questions are sent to pandas, where the numeric columns are aggregated correctly. The same file feeds both tracks. The router makes each query land in the right place.
